

KNr.	MNr.	Zuname, Vorname
Studium:	☐ Informatik (Bakk.)	☐ Elektrotechnik (Master)

Ges.)(100)

1.)(30)

2.)(25)

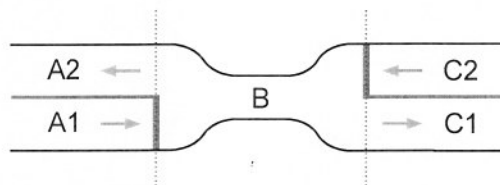
3.)(45)

Zusatzblätter:

Bitte verwenden Sie nur dokumentenechtes Schreibmaterial!

1 Synchronisation mit Semaphoren (30)

Der Fahrzeugverkehr an einer Straßenverengung (siehe Skizze) wird mit Hilfe von Prozessen simuliert. Die skizzierte Straße können Sie sich nach links und rechts beliebig verlängert vorstellen.



Das Codestück `car_left2right` simuliert die Fahrt eines von links, auf dem Fahrstreifen `A1` kommenden Autos, das die Engstelle `B` passiert und dann rechts, auf Fahrstreifen `C1` weiterfährt. Dabei stehen die Funktionsaufrufe `A1()`, `B()` und `C1()` für das Durchfahren der entsprechenden Straßenstücke. Analoges gilt für `car_right2left` und die Aufrufe von `C2()`, `B()` und `A2()`.

Ergänzen Sie die grau hinterlegten Felder der Codestücke mit Initialisierungscode bzw. mit Code zur Synchronisation der Fahrzeuge. Die Synchronisation der Fahrzeuge muss mittels *Semaphoren* erfolgen und folgende Anforderungen erfüllen:

- Im Bereich der Fahrbahnverengung (Abschnitt `B`) dürfen sich zu jedem Zeitpunkt nur Autos befinden, die in die selbe Richtung fahren.
- Im Abschnitt `B` dürfen sich maximal 3 Fahrzeuge gleichzeitig befinden.
- Befinden sich bereits Fahrzeuge im Abschnitt `B`, so werden neu ankommende Fahrzeuge, die Abschnitt `B` in die gleiche Richtung durchfahren wollen, wie die bereits in `B` befindlichen Fahrzeuge, bevorzugt gegenüber den Fahrzeugen, die in die Gegenrichtung fahren möchten.

Hinweis: Überlegen Sie, welche Teile der in der Vorlesung präsentierten Lösungen zu Synchronisationsaufgaben für die Lösung des hier gegebenen Synchronisationsproblems verwendbar sind.

Initialisierungen:

```
ltr = 0;  
rtl = 0;  
init(B1, 1);  
init(B2, 3);  
init(x1, 1);  
init(x2, 1);
```

void car_left2right(void)

{

A1();

wait(x1);

ltr++;

if (ltr == 1 then wait(B1);

signal(x1);

wait(B2);

B();

signal(B2);

wait(x1);

ltr--;

if (ltr == 0 then signal(B1);

signal(x1);

C1();

}

void car_right2left(void)

{

C2();

wait(x2);

rtl++;

if (rtl == 1 then wait(B1);

signal(x2);

wait(B2);

B();

signal(B2);

wait(x2);

rtl--;

if (rtl == 0 then signal(B1);

signal(x2);

A2();

}

2 Page Replacement (25)

Gegeben ist ein Arbeitsspeicher mit vier Frames, dessen Seiten mit unterschiedlichen Ersetzungsstrategien ersetzt werden sollen: mit OPT, LRU, bzw. mit dem Clock Algorithmus. Ergänzen Sie in den Tabellen für jeden Algorithmus die Speicherinhalte für jeden Frame nach jedem Zugriff der angegebenen Seitenzugriffsfolge an. Die Seitenzugriffsfolge ist für alle Algorithmen gleich. Sie ist jeweils in der Kopfzeile der Tabelle gegeben. Geben Sie in den Spalten die Speicherbelegung nach dem entsprechenden Seitenzugriff an und kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Der Arbeitsspeicher ist am Beginn leer. Für die ersten vier Speicherzugriffe sind die Tabellen bereits ausgefüllt.

OPT-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	E	E	E	E	E	E	E	E	E	E	E	E
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X				X							

LRU-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	F	F	F	F	F	F	F	F
1		B	B	B	B	B	B	B	B	B	B	D	D	D	D	D	D
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	E	E	E	E	E	E	E
PF	X	X	X	X		X			X	X	X	X					

Clock-Algorithmus:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	E	E	E	E	E	E	E	E
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X			X	X							

Vergleichen und diskutieren Sie die Anzahl der bei den Seitenersetzungsstrategien beobachteten Page Faults. Entspricht das Ergebnis dem zu erwartenden Verhalten?